

# Doorman — 온체인 재판매 상한 강제 티켓팅 프로토콜

Solana Summer School 캡스톤 프로젝트. 이 문서는 발표 자료 제작을 위해 문제 정의부터 기술적 해결 방식, 구현 현황까지 정리한 자료입니다. (다른 LLM에 붙여넣어 슬라이드/스크립트 초안을 뽑는 용도)

## 1. 문제 정의

콘서트·스포츠 경기 등 인기 이벤트 티켓은 정가로 풀리자마자 매크로/암표상이 선점해서, 실수요자는 **정가의 몇 배**를 주고 리셀 마켓에서 사야 하는 일이 반복된다.

기존 티켓팅 플랫폼(티켓링크, 인터파크, StubHub 등)이 이 문제를 다루는 방식의 근본적 한계:

- **가격 상한이 "정책"이지 "강제"가 아님.** 재판매 금지/상한 규정이 있어도 이는 플랫폼의 이용약관·서버 로직에 불과해서, 외부 리셀 채널(당근마켓, 트위터, 리셀 전문 사이트)로 새어나가는 거래를 원천적으로 막을 방법이 없다.
- **검증 불가능성.** "이 티켓이 정가에 팔렸는지, 상한 이내로 재판매됐는지"를 구매자가 스스로 검증할 방법이 없다 — 플랫폼의 내부 DB를 신뢰하는 수밖에 없다.
- **관할구역별 규제 대응이 수작업.** 국가/지역마다 암표 관련 법정 상한(예: 한국 공연법상 재판매 규제)이 다른데, 이를 이벤트마다 개별적으로 사람이 확인하고 반영해야 한다.
- **환불·재판매 정산이 불투명.** 얼마에 재판매됐는지, 언제 정산됐는지가 플랫폼 내부에 묻혀 있어 감사(audit)가 어렵다.

## 2. 해결 방향 — 핵심 아이디어

**"재판매 상한을 정책이 아니라 온체인 프로그램 로직으로 강제한다."**

티켓을 Solana 상의 SPL 토큰(Token-2022)으로 발행하되, **평상시엔 Frozen(전송 불가) 상태로 고정**해두고, 재판매가 이 프로그램의 `execute_resale` 명령어를 통해서만 — 그것도 사전에 검증된 가격 상한 이내에서만 — 일어나도록 만든다. 즉 티켓을 손에 쥔 순간부터 "이 티켓이 얼마 이상으로는 절대 거래될 수 없다"는 게 코드 수준에서 물리적으로 보장된다. 플랫폼을 신뢰할 필요 없이, 누구나 온체인에서 직접 검증 가능하다.

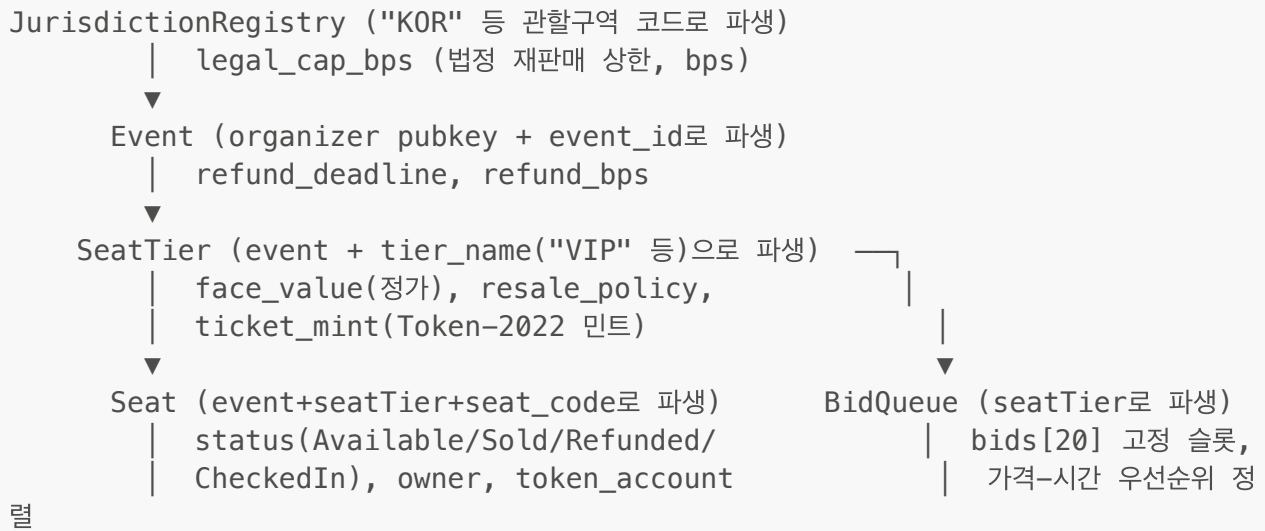
핵심 설계 결정 세 가지:

1. **가격 상한 = min(주최자 정책, 관할구역 법정 상한)** — 둘 중 더 엄격한 쪽이 항상 적용되도록 자동 계산 (`ResalePolicy::more_restrictive`). 주최자가 실수로 법정 상한보다 느슨한 정책을 걸어도 법정 상한이 우선한다.
2. **재판매는 "리스팅"이 아니라 "대기열 매칭"** — 판매자가 가격을 정해 올리는 방식이 아니라, 사려는 사람들이 먼저 상한 이내 금액으로 대기열(에스크로)에 예치해두고, 티켓 보유자가 `execute_resale`을 호출하면 **대기열 1순위와 원자적으로(atomic)** 체결된다. 판매자가 상한을 넘겨 부르고 싶어도 애초에 그런 가격을 받아줄 매수자가 대기열에 존재할 수 없다 (매수자 자신도 상한 이내로만 예치 가능).
3. **티켓 = 항상 Frozen 상태의 SPL 토큰** — 구매 시 발행되자마자 얼리고(freeze), 재판매/체크인 등 프로그램이 승인한 트랜잭션 내에서만 잠깐 녹였다가(thaw) 다시 열린다. 사용자가 지갑 앱에서 임의로 토큰을 전송해 상한을 우회하는 것 자체가 불가능하다.

## 3. 시스템 아키텍처

Anchor(Rust) 프로그램 하나(`ticketing-system`, program ID

`2vPTZ9iRydqA3mbkkK6CPXhFuTBdVc8s3otKmfcABiVK`)가 전체 로직을 담당한다. PDA(Program Derived Address) 계층 구조:



- **JurisdictionRegistry**: 관할구역별 법정 재판매 상한(bps)을 등록. 누구나(권한 있는 authority가) 생성 가능, 이벤트가 이걸 참조.
- **Event**: 주최자, 관할구역, 환불 마감일/환불 비율을 담는다. (이름/장소/이미지 같은 표시용 메타데이터는 온체인에 없음 — 의도적으로 최소 구조)
- **SeatTier**: 등급(VIP/R/S 등) 단위. 정가(face\_value), 실제 적용될 재판매 정책(**more\_restrictive**(주최자 정책, 법정 상한)), 그리고 그 등급 전용 Token-2022 티켓 민트를 가짐. 민트는 **DefaultAccountState=Frozen** 익스텐션으로 생성돼서, 이 민트로 발행된 모든 토큰 계정이 기본적으로 얼어있다.
- **Seat**: 개별 좌석. 상태 머신은 **Available** → **Sold** → {**Refunded**, **CheckedIn**} — 즉 한 번 팔리면 다시 **Available**로 되돌아갈 수 없다 (온체인 설계상 의도된 단방향성).
- **BidQueue**: 등급별 재판매 대기열. 고정 20슬롯 배열(데모 규모, 실서비스는 확장 필요)에 입찰을 가격 내림차순-동가일 땐 등록 순서로 정렬 유지. 예치금은 프로그램이 소유한 vault(SPL 토큰 계정)에 실제로 락업됨.

## 4. 핵심 트랜잭션 흐름

| 명령어                                       | 호출자          | 하는 일                                                                     |
|-------------------------------------------|--------------|--------------------------------------------------------------------------|
| <code>create_jurisdiction_registry</code> | 규제 authority | 관할구역 코드 + 법정 상한(bps) 등록                                                  |
| <code>create_event</code>                 | 주최자          | 이벤트 생성 (관할구역 연결, 환불 정책 설정)                                               |
| <code>create_seat_tier</code>             | 주최자          | 등급 생성 — 정가/재판매정책 설정, Token-2022 티켓 민트를 Frozen 기본값으로 생성, 대기열+vault 함께 초기화 |
| <code>create_seat</code>                  | 주최자          | 개별 좌석 계정 생성 (Available 상태로 시작)                                           |
| <code>buy_seat</code>                     | 고객           | 정가만큼 결제 토큰 이체 → 고객의 티켓 ATA를 thaw→mint 1개→다시 freeze. 좌석 상태 Sold로 전환       |
| <code>join_queue</code>                   | 대기 구매자       | 상한 이내 금액을 vault에 에스스로 → 대기열에 가격-시간 순으로 삽입                                |
| <code>leave_queue</code>                  | 대기 구매자       | 자기 입찰 취소, 예치금 전액 환불                                                      |

| 명령어                         | 호출자             | 하는 일                                                                                                                   |
|-----------------------------|-----------------|------------------------------------------------------------------------------------------------------------------------|
| <code>execute_resale</code> | 티켓 보유자<br>(판매자) | 대기열 1순위와 매칭 — 판매자는 1순위 입찰액을 받고, 구매자는 티켓(1 unit) 수령. 양쪽 ATA는 <code>thaw→transfer→freeze</code> 로 원자 처리. 대기열에서 매칭된 항목 제거 |
| <code>refund_ticket</code>  | 주최자             | 환불 마감일 이전이면 정해진 비율만큼 환불, 좌석 상태 <code>Refunded</code> 로 전환                                                              |
| <code>check_in</code>       | 주최자/상태<br>표     | 좌석 상태 <code>Sold</code> → <code>CheckedIn</code> (입장 처리)                                                               |

재판매 상한 공식:  $cap = face\_value \times (10000 + max\_bps) / 10000$  — 즉 정가 대비 몇 % 마크업까지 허용하는 방식 (정가의 몇 %가 아니라 "정가 + 마크업"). `join_queue`(입찰 시)와 `execute_resale`(체결 시) 양쪽에서 동일한 공식으로 검증해서 이중으로 강제된다.

## 5. 기술 스택

- **온체인:** Rust + Anchor 0.31, Solana Program, SPL Token-2022 (Frozen 기본 상태 익스텐션 활용)
- **테스트:** LiteSVM 기반 통합 테스트 — 상한 계산, 대기열 우선순위, 재판매 원자성, 환불 마감일 등 핵심 로직 검증
- **프론트엔드:** React 19 + Vite + TypeScript + TailwindCSS v4, [@coral-xyz/anchor](#) + [@solana/web3.js](#)로 devnet과 직접 통신 (지갑 확장 프로그램 없이, 브라우저 localStorage에 데모용 키페어 보관)
- **배포:** Solana devnet

## 6. 프론트엔드 데모 앱 (Doorman)

이벤트 카탈로그(홈), 이벤트 상세/좌석 예매, 내 티켓, 재판매, Admin(이벤트 생성/좌석·환불·재판매 현황 관리) 화면으로 구성된 SPA. 목록 카탈로그 이벤트들 중 **하나(데모 이벤트)만 실제 devnet 프로그램에 연결되어** 있어서, 화면에서 클릭하는 모든 액션(좌석 구매, 대기열 등록/취소, 재판매 체결)이 실제 devnet 트랜잭션으로 나가고, 토스트 알림에 Solana Explorer 링크가 즉시 뜬다. 여러 브라우저 계정(주최자/고객1~3)을 헤더에서 전환하며 매수자·매도자 역할을 오가는 멀티 액터 시나리오를 한 화면에서 시연할 수 있다.

## 7. 차별점 / 임팩트

- **신뢰가 필요 없는 검증 가능성(trustless verifiability):** "상한이 지켜진다"는 걸 플랫폼 말을 믿는 게 아니라 누구나 온체인 코드와 트랜잭션을 직접 확인할 수 있다.
- **구조적 방지 vs. 정책적 금지:** 기존 플랫폼은 "재판매 금지"를 약관으로 명시해도 외부 채널 유출을 막을 물리적 수단이 없다. 여기서 티켓 자체가 Frozen 토큰이라 프로그램이 승인한 경로 밖에서는 애초에 전송이 불가능하다.
- **관할구역 인지형 컴플라이언스:** 국가별 법정 상한을 온체인 레지스트리로 관리해서, 이벤트/등급 생성 시점에 **min(주최자 정책, 법정 상한)**이 자동 적용 — 사람이 지역별 규정을 일일이 대조할 필요가 없다.
- **원자적 정산:** 재판매 체결이 "티켓 이전 + 대금 지급 + 대기열 갱신"을 하나의 트랜잭션으로 처리해서 정산 리스크(먹튀, 이중매매)가 구조적으로 없다.

## 8. 현재 구현 상태 및 알려진 한계

**구현 완료:** 위 9개 명령어 전부 devnet에 배포·동작 확인 완료. 프론트엔드에서 구매→대기열→재판매→대기열취소까지 전체 플로우를 실제 트랜잭션으로 시연 가능.

**알려진 한계 (의도된 단순화 / 향후 과제):**

- 한 지갑은 같은 등급에서 평생 1좌석만 구매 가능. 티켓 ATA가 좌석이 아니라 등급(tier) 단위로 생성되고 **init** (재사용 불가)이라, 재판매로 넘겨서 다시 안 갖고 있어도 그 지갑은 해당 등급을 또 살 수 없다. 실서비스화 시 티켓 소유권 추적 모델 재설계 필요.
- 좌석은 한 번 팔리면 되돌릴 수 없음 (Sold → Refunded/CheckedIn만 가능, Available로 복귀 불가) — 환불도 "판매 취소"가 아니라 "상태 전환 + 대금 환불"이며 좌석 자체는 재판매 불가 상태로 남는다.
- 대기열 고정 20슬롯. **MAX\_QUEUE\_LEN=20**이 계정 크기에 하드코딩되어 있어, 실서비스에선 동적 확장 가능한 구조(별도 계정 체인, 힙 등)로 교체 필요.
- 이벤트 메타데이터(이름/장소/이미지)가 온체인에 없음. **Event** 계정은 organizer/eventId/jurisdiction/환불정책만 저장 — 표시용 정보는 현재 프론트엔드 목업으로만 존재. 실서비스화 시 별도 메타데이터 저장소(오프체인 인덱서 또는 온체인 확장) 필요.
- 티켓 양도(단순 전송) 명령어 없음. 현재는 구매/재판매(대기열 매칭)만 가능, "그냥 친구에게 선물" 같은 무상 이전 경로는 아직 없음.

**9. 향후 과제 (로드맵 제안)**

1. 등급당 다중 좌석 보유 지원 (티켓 소유권 모델을 계정당이 아닌 토큰 수량 기반으로 재설계)
2. 대기열 동적 확장 (고정 배열 → 별도 PDA 체인 또는 힙 구조)
3. 이벤트/좌석 메타데이터 온체인 또는 검증 가능한 오프체인 저장소 연동
4. 실지갑(Phantom 등) 연동 — 현재 데모는 브라우저 로컬 키페어
5. 메인넷 이전 전 보안 감사 (특히 Token-2022 freeze/thaw 권한 위임, PDA 서명 경계 검증)